

Desafio Técnico — Justarter

Desafio: Auto-completar na Busca

Olá! Esse desafio técnico tem como propósito medir suas habilidades, ver como estuda, pensa e se organiza na prática. O tempo de término é de 4 dias.

Após finalizar o desafio, nos envie um link para repositório do projeto no GitHub, GitLab ou BitBucket (o que preferir).

Registre tudo: testes que forem executados, ideias que gostaria de implementar se tivesse tempo (explique como você as resolveria), decisões tomadas e seus porquês, arquiteturas testadas e os motivos de terem sido modificadas ou abandonadas. Crie um arquivo `COMMENTS.md` ou `HISTORY.md` no repositório para registrar essas reflexões e decisões.

Dicas:

- Use ferramentas e bibliotecas open source, mas documente as decisões e porquês;
- Automatize o máximo possível;
- Em caso de dúvidas, pergunte.

Considerações Gerais

O problema aqui descrito tem algumas constraints:

- a) Preciso conseguir rodar seu código em um **Ubuntu** ou **Mac OS X**;
- b) Queremos rodar o projeto de maneira simples. Recomendamos que o comando `docker compose up` seja capaz de subir todas as partes do desafio (front, back...)

O arquivo contém imagens de exemplo para a parte Web — são apenas referências, sinta-se livre para criar sua própria interface.

Planejamento do Projeto

Antes de começar a codar, quebre o projeto em tarefas menores (microtasks) e documente essa divisão em um arquivo `TASKS.md`. Não existe um formato único esperado — se você já usa alguma metodologia de planejamento assistido por IA, sinta-se à vontade para aplicá-la aqui.

O que queremos entender:

- Como você decompôs o problema em partes menores;
- Se a ordem das tarefas faz sentido;
- Se o planejamento se refletiu na forma como os PRs foram organizados depois.

Fluxo de Pull Requests

Todas as mudanças devem ser submetidas via Pull Request — não faça push direto na branch principal (**main**). Isso vale mesmo trabalhando sozinha: o objetivo é exercitar o mesmo fluxo de revisão que usamos no dia a dia aqui no Jusbrasil.

Sugestão: se você documentou microtasks no **TASKS.md**, tente refletir essa divisão nos PRs — PRs menores e coerentes com uma tarefa específica facilitam a leitura.

CI

Configure uma esteira de CI no repositório (GitHub Actions, GitLab CI, ou equivalente) que rode automaticamente a cada PR, checando ao menos lint e testes. Fique à vontade para evoluir o CI além disso, se quiser.

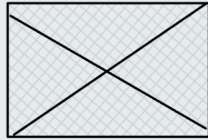
O Desafio

Você deve implementar uma única página com um formulário de busca onde, ao digitar o termo inicial, sugestões serão exibidas para completar o termo. Seu sistema deve rodar na Web utilizando ReactJS no front-end + backend na linguagem de sua preferência. O front-end obrigatoriamente deve se comunicar com um GraphQL, e o GraphQL é quem deve se comunicar com o back-end.



Busca

A página de busca consiste em pelo menos um campo para a pessoa usuária preencher o que deseja buscar. A página precisa ser responsiva, exibida amigavelmente em dispositivos móveis. Use as imagens de referência do repositório como guia — mas sinta-se livre para criar sua própria interface.



Busca com Autocompletar

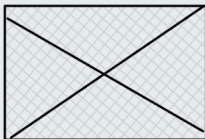
Digite no campo abaixo para exibir as sugestões

BUSCAR

Sugestões do Autocompletar

A exibição do autocompletar deve atender aos seguintes requisitos:

- As sugestões só começam a ser exibidas após digitar, no mínimo, 4 caracteres;
- Se não existem sugestões para o termo digitado, nenhum elemento deve ser exibido abaixo do campo de busca;
- O backend precisa retornar, no máximo, 20 sugestões — porém somente 10 são exibidas; as demais o usuário acessa via scroll dentro do elemento do auto-completar;
- As sugestões precisam manter em **negrito** a parte do termo que corresponde ao termo inicial da busca;
- Ao passar o mouse por alguma sugestão (hover) ou tocar no elemento (touch no mobile), o elemento deve ser destacado;
- O usuário pode continuar digitando — as sugestões vão mudando dinamicamente conforme o termo é atualizado;
- As sugestões precisam ser exibidas na mesma (ou próxima) velocidade que o usuário digita;
- Ao clicar em alguma sugestão, o campo de busca principal deve ser atualizado com o texto da sugestão.



Busca com Autocompletar

Digite no campo abaixo para exibir as sugestões

- direito do consumidor
- direito do trabalho
- direito do consumidor e dano moral
- direito do trabalho insalubridade metalurgico
- direito do consumidor inversao do ônus da prova
- direito dos herdeiros de exigir as contas
- direito dos animais

Lista de Sugestões / Persistência

O armazenamento das sugestões fica a seu critério — banco de dados open source, arquivo JSON populado na pasta do projeto, ou outra solução. Os termos sugeridos também ficam a seu critério: prepare uma lista que atenda a um domínio de sua preferência.

Se optar por banco de dados, forneça um meio automatizado de populá-lo — não precisamos cadastrar sugestões manualmente para testar o projeto.

Código de Referência 🔍

O repositório inclui um arquivo `starter/suggestions.js` com uma implementação inicial do handler de sugestões. Você não é obrigada a usá-lo — mas pedimos que o avalie no seu `COMMENTS.md` antes de decidir:

- O que está correto e faria sentido manter?
- O que você faria diferente, e por quê?
- Tem alguma coisa que você mudaria antes de levar esse código para produção?

Não há resposta certa nem errada — o que queremos entender é como você lê e pensa sobre código que não foi você quem escreveu. Uma observação com boa justificativa vale mais do que uma lista longa sem explicação.

O arquivo `starter/suggestions.js`:

JavaScript

```
// starter/suggestions.js
//
// Implementação inicial de referência para o handler de sugestões.
// Você não é obrigada a usá-la – mas avalie este código no seu
// COMMENTS.md antes de decidir: o que está bom, o que mudaria e por quê.

async function fetchSuggestions(query, setSuggestions) {
  if (query.length < 4) {
    setSuggestions([]);
    return;
  }

  const response = await fetch(`/graphql?q=${query}`, {
    method: "GET",
  });

  const data = await response.json();
  setSuggestions(data.suggestions.slice(0, 10));
}

export default fetchSuggestions;
```

Uso de Inteligência Artificial

No Jusbrasil, IA faz parte do dia a dia de quem desenvolve software — por isso você está livre para usar as ferramentas de IA que preferir (Claude, ChatGPT, Copilot, Cursor, ou qualquer outra) durante todo o desafio, do planejamento ao código. Não tem pegadinha aqui: o que queremos entender não é se você "sabe usar" uma ferramenta, mas como você pensa enquanto a usa.

O que é obrigatório registrar no **COMMENTS.md/HISTORY.md**:

Sempre que usar IA de forma relevante para uma decisão de código (não precisa registrar cada autocomplete de linha, mas sim decisões de arquitetura, trechos de lógica mais complexos, ou qualquer coisa que tenha gerado dúvida), registre:

- **O que você pediu** — o prompt ou a pergunta feita à ferramenta;
- **O que você aceitou como veio** — e por quê fazia sentido;
- **O que você alterou** — e o motivo da alteração;
- **O que você rejeitou** — sugestões que a IA deu e você decidiu não usar, e por quê.

O que não esperamos:

- Que você não use IA — usar é parte natural do processo;

- Que você documente cada interação trivial;
- Que você tenha um histórico "perfeito" sem nenhuma correção — pelo contrário, registrar onde você corrigiu a IA é um sinal positivo, não negativo.

O que de fato avaliamos:

- Se há evidência de que você revisou e entendeu o que foi gerado, em vez de aceitar tudo sem checar;
- Se as decisões registradas mostram raciocínio próprio, mesmo quando a IA ajudou a chegar lá;
- Coerência entre o que está documentado aqui e o que você consegue explicar sobre o seu próprio código depois.

Avaliação do Desafio

Alguns pontos que serão analisados:

- Atende as funcionalidades propostas (auto-completar uma busca);
- Organização do código;
- Facilidade ao rodar o projeto;
- Justificativa das decisões no arquivo `COMMENTS.md` ou `HISTORY.md`;
- Planejamento e organização em microtasks (refletido no `TASKS.md` e na granularidade dos PRs);
- Uso do fluxo de Pull Requests;
- Qualidade do uso de IA registrado — capacidade de validar, ajustar e justificar o que foi gerado, e não a quantidade ou sofisticação do uso.